

SIMATS ENGINEERING
Department of Computer Science and
Engineering ASSESSMENT TOOL2 -CONCEPT MAPPING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO1	Assessment:	ASSESSMENT TOOL 2- CONCEPT MAPPING
Weightage:	35%	Total Marks:	25
CO1 Statement:	Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking(BL4)		
Student Name:	K.Aristotle	Reg.No.:	192511196
Department:	CSE	Date:	

ANNEXURE A

1. Construct a detailed concept map showing the relationship between CPU (ALU, Control Unit, Registers), memory (cache, RAM), and I/O devices. Extend the map to include single-bus, multi-bus, and hierarchical bus structures, and clearly illustrate how data and control signals flow among these components during execution. Indicate possible points of delay or bottlenecks and how different bus structures help in improving performance.

1. Basic Concept

A computer system mainly consists of CPU, Memory and I/O devices.

2. CPU

The CPU (Central Processing Unit) executes instructions.

- ALU: Performs arithmetic and logical operations.
- Control Unit: Controls and coordinates all operations.
- Registers: Store temporary data and instructions.

3. Memory and I/O

- Cache: Fast memory that stores frequently used data near the CPU.

- RAM: Stores programs and data currently being processed.
- I/O devices: Allow communication with the computer, such as keyboard, monitor and printer.

4. Bus Structures

Single Bus:

All CPU, memory and I/O devices share one bus. It is simple but can cause delay and bottleneck because only limited transfers can occur at a time.

Multi-Bus:

Uses multiple buses, allowing different data transfers simultaneously. Therefore, it reduces congestion and improves speed.

Hierarchical Bus:

Uses different levels of buses. High-speed buses connect CPU and memory, while slower buses connect I/O devices. This improves overall performance.

5. Data and Control Flow

During execution:

CPU → Address Bus → Memory → Data Bus → CPU

The Control Bus carries signals such as Read, Write and Interrupt.

Conclusion:

Multi-bus and hierarchical bus structures reduce bottlenecks and provide faster and more efficient communication than a single-bus structure.

2. Develop a comprehensive concept map linking the instruction execution cycle (fetch, decode, execute) with CPU performance parameters such as CPI, clock cycles, and execution time. Show how each stage contributes to overall execution time, and include connections to factors like memory access, instruction complexity, and pipeline stalls. Also, represent how benchmarking is used to evaluate performance.

The instruction execution cycle consists of three main stages:

1. Fetch – The CPU fetches the instruction from memory. Cache or RAM access time can affect this stage.
2. Decode – The Control Unit decodes the instruction and identifies the required operation.
3. Execute – The ALU or other CPU units perform the required operation.

CPU Performance Parameters

- CPI (Cycles Per Instruction): Average number of clock cycles required to execute one instruction.

- Clock Cycle: Time required for one CPU clock cycle.
- Execution Time: Total time required to execute a program.

Formula:

Execution Time = Instruction Count $\times$ CPI $\times$ Clock Cycle Time

Factors Affecting Performance

- Memory Access: Slow RAM or cache misses increase execution time.
- Instruction Complexity: Complex instructions may require more clock cycles.
- Pipeline Stalls: Data, control and structural hazards can stop the pipeline and increase clock cycles.

Benchmarking

Benchmarking is the process of testing a CPU using standard programs or workloads. It measures factors such as execution time, CPI and overall performance to compare different CPUs.

Conclusion: Lower CPI, fewer clock cycles and lower execution time generally result in better CPU performance.

3. Create a concept map comparing 8085 and 8086 architectures, including their instruction sets, register organization, addressing modes, and memory segmentation (in 8086). Clearly show how these architectural features influence program execution, multitasking capability, and efficiency in real-time applications.

8085 and 8086 Architecture

8085 and 8086 are microprocessors with different architectural features.

8085

- 8-bit microprocessor.
- Has a 16-bit address bus and can access 64 KB memory.
- Registers include Accumulator, B, C, D, E, H, L, Stack Pointer, Program Counter and Flag register. • Has a simple instruction set.
- Supports basic addressing modes such as Immediate, Register and Direct.
- Suitable for simple control and embedded applications.

8086

- 16-bit microprocessor.
- Has a 20-bit address bus and can access 1 MB memory.
- Registers include AX, BX, CX, DX, SP, BP, SI and DI.
- Has a larger and more powerful instruction set.

- Supports more addressing modes such as Immediate, Register, Direct, Indirect, Based and Indexed.
- Uses memory segmentation.

Memory Segmentation in 8086

Memory is divided into different segments:

- Code Segment (CS) – stores instructions.
- Data Segment (DS) – stores data.
- Stack Segment (SS) – stores stack data.
- Extra Segment (ES) – provides additional data storage.

Effect on Performance

- 8085: Simple architecture provides easy execution but limited performance.
- 8086: More registers, powerful instructions and segmentation improve execution speed and memory utilisation.
- 8086 supports more complex programs and better multitasking.
- These features make 8086 more suitable for efficient real-time applications.

4. Draw a detailed concept map that connects various addressing modes (immediate, direct, register, indirect, indexed) with instruction types (data transfer, arithmetic, control instructions). Illustrate how each addressing mode affects memory access time, instruction size, and execution speed, and show real-world scenarios where specific addressing modes are preferred.

Effect of Addressing Modes

- Immediate Addressing: Operand is present in the instruction itself. It requires little or no extra memory access, so execution is fast. Preferred for constants and initial values.
- Direct Addressing: The memory address of the operand is directly given. It requires memory access and usually has a larger instruction size. Useful for accessing fixed variables.
- Register Addressing: Operand is stored in a CPU register. It has very fast execution because no memory access is required. Preferred for frequent calculations.
- Indirect Addressing: A register contains the address of the operand. It requires memory access, so it is generally slower than register addressing. Useful for pointers and dynamic data.
- Indexed Addressing: Effective address is obtained using a base address and index register. It may require memory access but is very useful for arrays, tables and strings.

Real-World Applications

- Immediate: Loading a fixed value such as 5.
- Direct: Accessing a fixed variable stored in memory.
- Register: Performing repeated arithmetic calculations.
- Indirect: Accessing data through pointers.
- Indexed: Accessing elements of an array or table.

Conclusion: Register and immediate addressing generally provide faster execution, while indirect and indexed addressing provide greater flexibility for accessing memory and complex data structures.

5. Prepare an elaborate concept map integrating number systems (binary and hexadecimal) with instruction representation, memory storage, and debugging processes. Show how data is converted between formats, how instructions are stored and interpreted by the CPU, and why hexadecimal representation is commonly used in low-level programming and system design.

Conversion Between Formats

- Binary → Hexadecimal: Group binary digits into groups of 4 bits.
- Hexadecimal → Binary: Convert each hexadecimal digit into 4 binary bits.
- Example: Binary 1010 → Hexadecimal A • Example: Hexadecimal 2F → Binary 0010 1111

Instruction Representation

Instructions are stored in memory as binary machine code. The CPU:

1. Fetches the instruction from memory.
2. Decodes the binary instruction.
3. Executes the required operation.

Memory Storage

- Computers store instructions and data internally as binary (0 and 1).
- Each instruction and data value occupies one or more memory locations. • Hexadecimal is used to represent these binary values in a shorter form.

Debugging

Hexadecimal and binary values help programmers:

- Inspect memory addresses.
- Check register contents.
- Identify incorrect machine instructions. • Find memory and data errors.

Why Hexadecimal is Commonly Used

- One hexadecimal digit represents exactly 4 binary bits.
- It is much shorter and easier to read than long binary numbers.
- It makes memory addresses, machine code and register values easier to understand.
- Therefore, hexadecimal is widely used in assembly programming, debugging and system design.

Code of Conduct:

I c.abhinay adithya(192572115) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: c.abhinay adithya

MARKS ALLOCATION

Criteria	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence

Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand
---------	-----	--	---	---	---